

DEEP REINFORCEMENT LEARNING

365.382 (SS26)

Introduction to Reinforcement Learning



Sohvi Luukkonen

LIT AI LAB

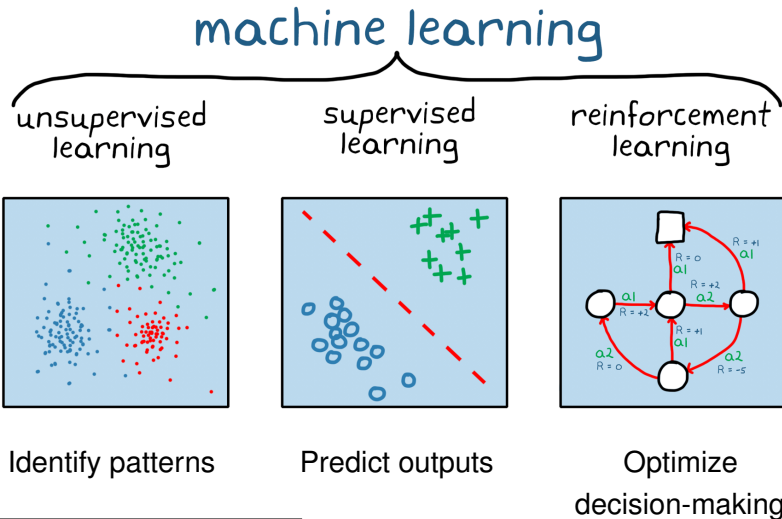
Institute for Machine Learning

Recap & Resources

- This is a recap to review the main/basic concepts
- Basic concepts should be clear after this recap.
- If not, you should invest some additional time to catch up
- [Reinforcement Learning: An Introduction](#) [Chapters 1 to 3]
- [DeepMind x UCL | RL Lecture Series 2021](#) [Lectures 1 to 3]

What is Reinforcement Learning?

Types of Machine Learning



1

¹<https://uk.mathworks.com/discovery/reinforcement-learning.html>

What is Reinforcement Learning?

Computational approach of **learning to make decisions** from **interactions**.

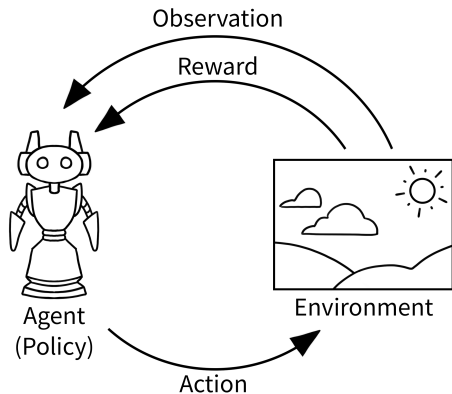
Characteristics of RL

- No labeled targets, only scalar **reward** signal
- **Delayed feedback**, not instantaneous – credit assignment
- **Active** data collection – actions influence future states and data distributions
- **Sequential** interactions – long-term dependencies
- **Goal-directed**: maximize cumulative reward

The Reward Hypothesis

All goals can be described by the maximization of expected cumulative reward

The Interaction Loop: Agent and Environment



At each step t

■ the agent:

- ☐ Executes action A_t
- ☐ Receives observation O_t
- ☐ Receives scalar reward R_t

■ the environment:

- ☐ Receives action A_t
- ☐ Emits observation O_{t+1}
- ☐ Emits scalar reward R_{t+1}

Goal: maximize cumulative reward (called the **return**)

$$G_t = R_t + R_{t+1} + R_{t+2} \dots$$

Formalizing the RL Interaction

Refresher: Expected Values

$$\mathbb{E}[X] \doteq \sum_{x_i} x_i \cdot \Pr\{X = x_i\}$$

$$\mathbb{E}[X|Y = y_j] = \sum_{x_i} x_i \cdot \Pr\{X = x_i|Y = y_j\}$$

$$\mathbb{E}[X|Y = y_j] = \sum_{z_k} \Pr\{Z = z_k|Y = y_j\} \cdot \mathbb{E}[X|Y = y_j, Z = z_k]$$

$$\mathbb{E}[f(X)] = \sum_{x_i} f(x_i) \cdot \Pr\{X = x_i\}$$

Markov Decision Process

Definition (Markov Decision Process)

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma \rangle$, where

- $\mathcal{S}$ is the set of all **states**
- $\mathcal{A}$ is the set of all **actions**
- $\mathcal{P}$ are the **joint transition probabilities** with **Markov property**
 - $p(r, s' | s, a)$ is the joint transition probability of a reward r and next state s' , given state s and action a
 - p defines the **dynamics** of the problem
- $\gamma \in [0, 1]$ is a **discount factor** weighting near vs. far future rewards

Markov Decision Process (Alternative Definition)

Definition (Markov Decision Process)

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where

- $\mathcal{S}$ is the set of all **states**
- $\mathcal{A}$ is the set of all **actions**
- $\mathcal{P}$ are the **state transition probabilities** with **Markov property**
 - $p(s'|s, a) = \sum_r p(s', r|s, a)$ is the transition probability of next state s' , given state s and action a
 - p defines the **dynamics** of the problem
- $\mathcal{R}$ is a **reward** function
 - $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the **expected reward**, achieved on a transition starting in (s, a) :
$$r(s, a) = \mathbb{E}[R|s, a] = \sum_r r \sum_{s'} p(s', r|s, a)$$
- $\gamma \in [0, 1]$ is a **discount factor** weighting near vs. far future rewards

Markov Property

"The future is independent of the past given the present"

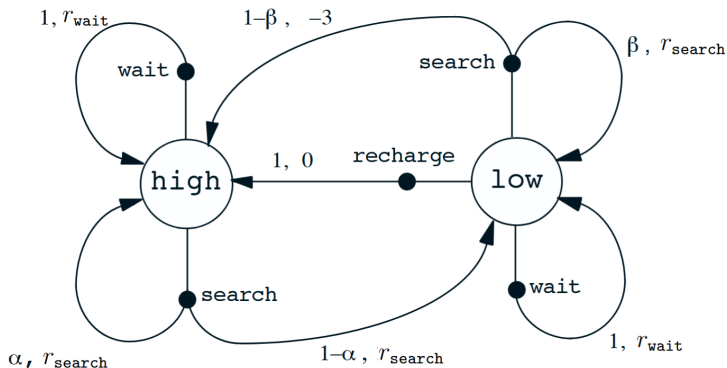
Definition (Markov Property)

A state representation is **Markov** if the current state contains all relevant information needed to predict the next state and reward.

$$P(\underbrace{S_{t+1}, R_{t+1}}_{\text{future}} \mid \underbrace{S_t, A_t}_{\text{present}}, \underbrace{S_{t-1}, A_{t-1}, \dots}_{\text{past}}) = P(\underbrace{S_{t+1}, R_{t+1}}_{\text{future}} \mid \underbrace{S_t, A_t}_{\text{present}})$$

Example: Cleaning Robot

- States (low and high battery): $\mathcal{S} = \{\text{low}, \text{high}\}$
- Actions: $\mathcal{A}(\text{low}) = \{\text{wait}, \text{search}, \text{recharge}\}$ and $\mathcal{A}(\text{high}) = \{\text{wait}, \text{search}\}$
- Rewards: $r_{\text{search}} > r_{\text{wait}}$
- Stochastic transition probabilities



Formalizing the Objective

Discounted Returns

Definition (Discounted Returns)

The discounted **returns** G_t is the sum of discounted future rewards:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) = R_{t+1} + \gamma G_{t+1} \end{aligned}$$

- $\gamma \in [0, 1)$ makes the sum finite
- $\gamma \rightarrow 0$ only immediate reward will be considered
- $\gamma \rightarrow 1$ rewards far in the future are taken into account more strongly
- recursive formulation

Policies and Goal of an RL Agent

Definition (Policy)

A **policy** π is a mapping $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ that assigns to each state–action pair (s, a) the **probability of selecting action a in state s** . It is denoted by $\pi(a|s)$, with the constraint $\sum_{a' \in \mathcal{A}} \pi(a'|s) = 1, \forall s \in \mathcal{S}$.

- A policy fully defines the behavior of an agent
- MDP policies *only* depend on the current state
- Can be deterministic or stochastic

Definition (Goal of an RL Agent)

To find a behavior policy π that maximizes the (**expected**) return G_t

Value Functions

Definition

The **state-value function** $v_{\pi}(s)$ of an MDP is the expected return starting from the state s , and then following the policy π :

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s]$$

The **action-value function** $q_{\pi}(s, a)$ of an MDP is the expected return starting from the state s , taking action a , and then following the policy π :

$$q_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a]$$

$$v_{\pi}(s) = \sum_a \pi(a|s) q_{\pi}(s, a) = \mathbb{E}[q_{\pi}(S_t, A_t) | S_t, \pi], \forall s$$

Optimal Value Functions

Definition

The **optimal state-value function** $v^*(s)$ is the maximum state-value function over all policies:

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

The **optimal action-value function** $q^*(s, a)$ is the maximum action-value function over all policies:

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

- The optimal value function gives the best possible performance in the MDP.
- The MDP is "solved" when the optimal value function is known.

Optimal Policy

Policies can be compared via their value functions:

$$\pi \geq \pi' \quad \text{if} \quad v_{\pi}(s) \geq v_{\pi'}(s), \forall s$$

Theorem (Optimal Policies)

For any MDP

- *There exists an optimal policy π^* that is better than or equal to all other policies, $\pi^* \geq \pi, \forall \pi$*
- *All optimal policies achieve the optimal state-value function, $v_{\pi^*}(s) = v^*(s)$*
- *All optimal policies achieve the optimal action-value function, $q_{\pi^*}(s, a) = q^*(s, a)$*

Finding an Optimal Policy

An optimal policy can be found by maximizing over $q^*(s, a)$,

$$\pi^*(s, a) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_a q^*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

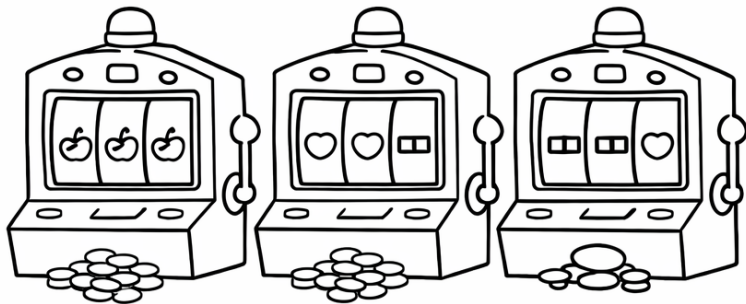
- There is always a **deterministic optimal policy** for any MDP
- If we **know** $q^*(s, a)$, we immediately have the **optimal policy**
- There can be **multiple** optimal policies
 - If multiple actions maximize $q^*(s, \cdot)$, we can (stochastically) pick any of these

Exploration vs. Exploitation

- Learning agents must balance two competing objectives:
 - **Exploitation** – maximize performance based on current knowledge
 - **Exploration** – increase knowledge
- Short-term suboptimal actions can improve long term return

Multi-Armed Bandit

k -armed Bandits



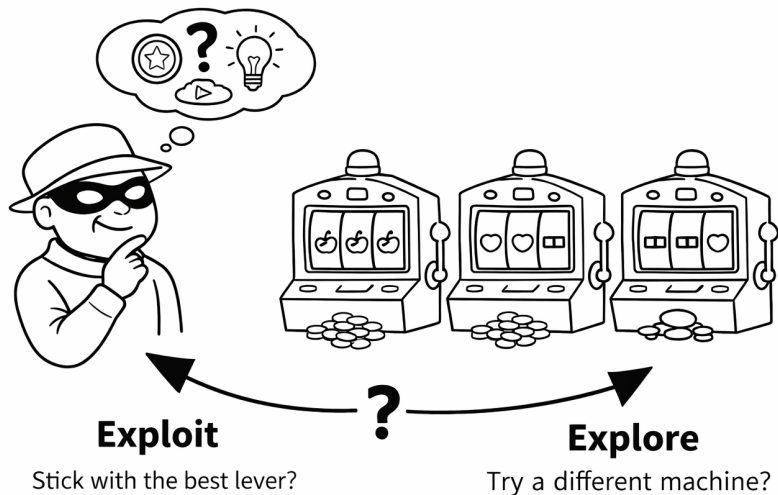
How to **maximize winnings** in the long run?

Find the **most rewarding** arm.

k -armed Bandits

- Only a **single state** (non-associative setting)
 - actions do not have long-term consequences in the environment
 - actions still impact **immediate reward**
- Repeated choice **multiple actions** (k choices)
- After each action, we get a **numeric reward** (drawn from the arm's **unknown stationary probability distribution**)

Learn How to Act



Values and Regret

- The **action value** for an action a is the expected reward

$$q(a) = \mathbb{E}[R_t | A_t = a]$$

- The **optimal value** is

$$q^* = \max_a q(a) = \max_a \mathbb{E}[R_t | A_t = a]$$

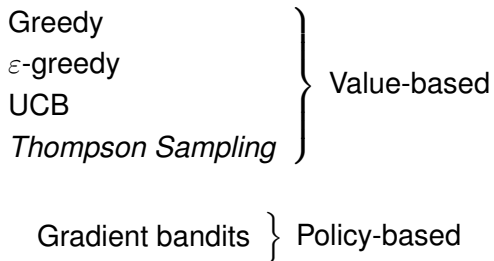
- **Regret** of an action is

$$\Delta_a = q^* - q(a)$$

- Maximize cumulative reward $\equiv$ minimize **total regret**:

$$L_t = \sum_{n=1}^t q^* - q(A_n) = \sum_{n=1}^t \Delta_{A_n}$$

Algorithms



Approximating Action Values

The **true action value** of an action a is the expected reward:

$$q(a) = \mathbb{E}[R_t | A_t = a]$$

A simple **estimate** is the averaged of sampled rewards:

$$Q_t(a) = \frac{\sum_{n=1}^t R_n \cdot \mathbf{1}_{A_n=a}}{\sum_{n=1}^t \mathbf{1}_{A_n=a}}$$

The **count** for action a is

$$N_t(a) = \sum_{n=1}^t \mathbf{1}_{A_n=a}.$$

By law of large numbers, Q_t converges to q , if each actions is selected at least occasionally, as $t \rightarrow \infty$.

Incrementally Approximating Action Values

We can also do the updates iteratively:

$$Q_t(A_t) = Q_{t-1}(A_t) + \underbrace{\alpha_t (R_t - Q_{t-1}(A_t))}_{\text{predicted error}},$$

$$\forall a \neq A_t : Q_t(a) = Q_{t-1}(a)$$

with $\alpha_t = \frac{1}{N_t(A_t)}$ and $N_t(A_t) = N_{t-1}(A_t) + 1$, where $N_0(a) = 0$.

- Other **step sizes** α are possible
- For example, constant α would lead to **tracking** instead of averaging (desirable for non-stationary reward distributions).

This is a general form of the update rule that will occur frequently in this lecture:

$$\textit{New} \leftarrow \textit{Old} + \textit{Stepsize}[\textit{Target} - \textit{Old}]$$

Greedy Policy

Select action with highest value: $A_t = \operatorname{argmax}_a Q_t(a)$

- Can get stuck on a suboptimal action forever
- Linear expected total regret

ϵ -greedy Policy

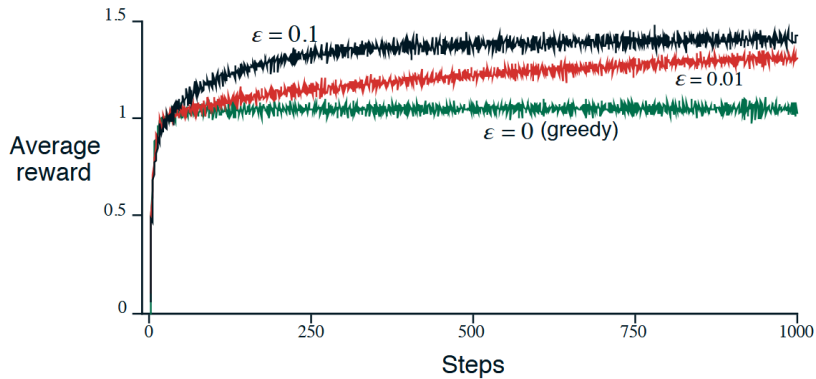
"Do something random, once in while"

- With probability $1 - \epsilon$ chose greedy action: $A_t = \operatorname{argmax}_a Q_t(a)$
- With probability ϵ select random action

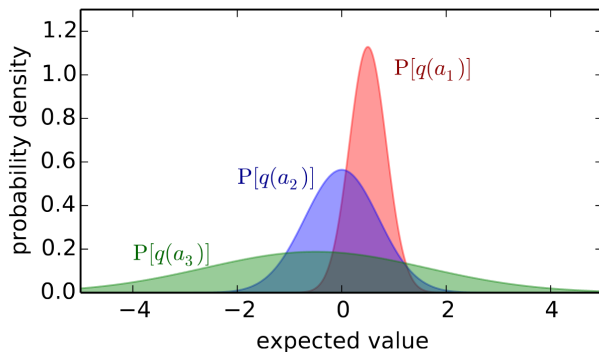
ϵ -greedy continues to explore and ensures that in the $t \rightarrow \infty$ limit all actions are selected infinitely many times.

- Guarantees that $Q_t(a)$ will converge to $q(a)$.
- ϵ -greedy with constant ϵ has linear expected total regret

Greedy vs. ϵ -greedy



Exploring Under Uncertainty



- Which action to pick?
- Actions with high uncertainty are important to explore than actions we are confident about.

Upper Confidence Bounds

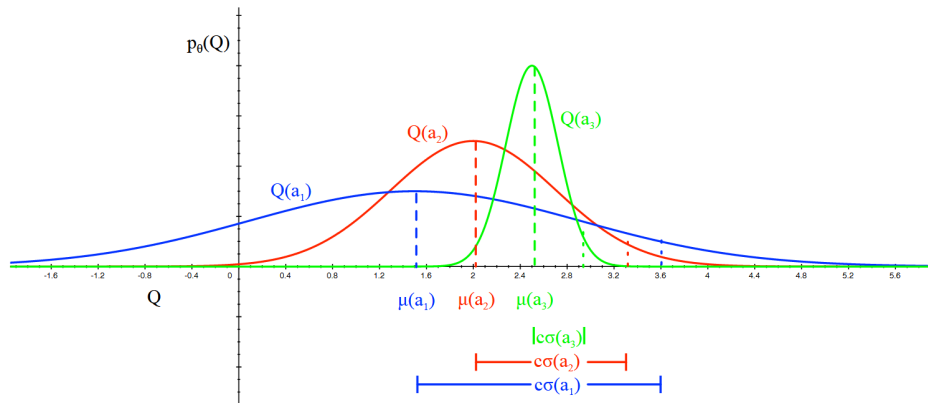
UCB selects actions as

$$A_t = \operatorname{argmax}_a \underbrace{Q_t(a)}_{\text{action quality}} + c \underbrace{\sqrt{\frac{\log t}{N_t(a)}}}_{\text{uncertainty}},$$

with $N_t(a)$ the number of times action a has been selected and $c > 0$ control the degree of exploration.

- selecting a increases $N_t(a)$ and decreases uncertainty
- selecting other actions, only increases t and increases uncertainty (magnitude of this effect gets smaller over time due to $\log(\cdot)$)

Upper Confidence Bounds



Derivation of UCB (I)

Theorem (Hoeffding's inequality)

Let $X_1, \dots, X_n$ be independent random variables with values in $[0, 1]$. Let $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ and $\mu = \mathbb{E}[X_i]$. Then, for all $u > 0$,

$$p(\bar{X} + u \leq \mu) \leq e^{-2nu^2}.$$

- Apply to bandits with bounded rewards $R_t \in [0, 1]$

$$p(Q_t(a) + U_t(a) \leq q(a)) \leq e^{-2N_t(a)U_t(a)^2}$$

- By symmetry:

$$p(Q_t(a) - U_t(a) \geq q(a)) \leq e^{-2N_t(a)U_t(a)^2}$$

Derivation of UCB (II)

- We want a high-probability upper bound on $q(a)$
- Set failure probability δ_t and solve:

$$e^{-2N_t(a)U_t(a)^2} = \delta_t$$

- Taking logs:

$$-2N_t(a)U_t(a)^2 = \ln \delta_t$$

- Rearranging:

$$U_t(a) = \sqrt{\frac{\ln(1/\delta_t)}{2N_t(a)}}$$

- Choose time-dependent confidence $\delta_t = t^{-4}$:

$$U_t(a) = \sqrt{\frac{2 \ln t}{N_t(a)}}$$

Derivation of UCB (III)

- Upper confidence bound:

$$\text{UCB}_t(a) = Q_t(a) + \sqrt{\frac{2 \ln t}{N_t(a)}}$$

- Action selection:

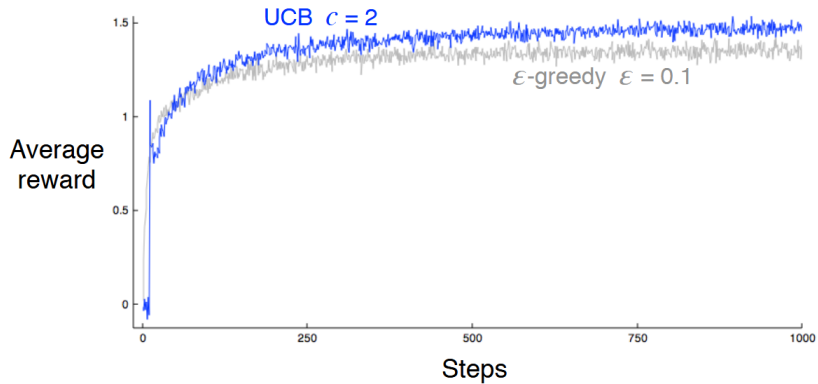
$$A_t = \arg \max_a \left(Q_t(a) + \sqrt{\frac{2 \ln t}{N_t(a)}} \right)$$

- Interpretation:

- ☐ $Q_t(a)$: exploitation (high reward)
- ☐ $\sqrt{\frac{2 \ln t}{N_t(a)}}$: exploration (uncertainty)
- ☐ Rarely selected arms $\Rightarrow$ large bonus

- Result: logarithmic regret $O\left(\sum_a \frac{\ln T}{\Delta_a}\right)$

ϵ -greedy vs. UCB



Gradient Bandits

Instead of learning values, learn directly a policy $\pi(a)$ by learning **action preferences** $H_t(a)$:

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}} \quad (\text{softmax})$$

- Preferences are not values – just learnable policy **parameters**
- Actions with **higher relative preferences** are taken more often

Policy Gradients

Update policy parameters such that **expect value increases** using **gradient ascent**:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \mathbb{E}[R_t | \pi_{\theta_t}]$$

Log-likelihood trick (also known as REINFORCE trick, Williams 1992):

$$\nabla_{\theta} \mathbb{E}[R_t | \pi_{\theta_t}] = \mathbb{E}[R_t \nabla_{\theta} \log \pi_{\theta}(A_t)]$$

We can sample this, and so the **stochastic gradient ascent** on the (true) value of the policy is

$$\theta_{t+1} = \theta_t + \alpha R_t \nabla_{\theta} \log \pi_{\theta}(A_t)$$

(we are using sampled rewards – does not use value estimates)

Gradient Bandits

For action preferences:

$$\begin{aligned}H_{t+1}(a) &= H_t(a) + \alpha R_t \frac{\partial \log \pi_t(A_t)}{\partial H_t(a)} \\&= H_t(a) + \alpha R_t (\mathbf{1}_{A_t=a} - \pi_t(a))\end{aligned}$$

$\Rightarrow$

$$\begin{aligned}H_{t+1}(A_t) &= H_t(A_t) + \alpha R_t (1 - \pi_t(A_t)) \\H_{t+1}(a) &= H_t(a) - \alpha R_t \pi_t(a), \quad \forall a \neq A_t\end{aligned}$$

- Preferences of actions with higher rewards increase more, making them more likely to be selected again
- Can get stuck in to local optima, and not reach the optimal policy

Derivation of Gradient Bandits

Start from

$$H_{t+1}(a) = H_t(a) + \alpha R_t \frac{\partial \log \pi_t(A_t)}{\partial H_t(a)},$$

with softmax policy

$$\pi_t(i) = \frac{e^{H_t(i)}}{\sum_j e^{H_t(j)}}.$$

Then

$$\begin{aligned} \log \pi_t(A_t) &= H_t(A_t) - \log \sum_j e^{H_t(j)}, \\ \frac{\partial \log \pi_t(A_t)}{\partial H_t(a)} &= \mathbf{1}_{A_t=a} - \frac{e^{H_t(a)}}{\sum_j e^{H_t(j)}} = \mathbf{1}_{A_t=a} - \pi_t(a). \end{aligned}$$

Plugging back gives

$$H_{t+1}(a) = H_t(a) + \alpha R_t (\mathbf{1}_{A_t=a} - \pi_t(a)).$$

RL Taxonomy

RL Taxonomy 1

■ Value-based

- ☐ Learned value function
- ☐ Implicit policy (ϵ -greedy)

■ Policy-based

- ☐ Learned policy
- ☐ No value function

■ Actor-Critic

- ☐ Learned policy (actor)
- ☐ Learned value function (critic)

RL Taxonomy 2

■ On-policy RL

- ☐ Learn π with samples from π

■ Off-policy RL

- ☐ Learn π with samples from μ

■ Offline RL

- ☐ Learn π from pre-collected samples from (unknown) μ

RL Taxonomy 3

■ Model-free

- ☐ Policy and/or value function
- ☐ No model

■ Model-based

- ☐ Policy and/or value function
- ☐ Model

RL Taxonomy

